



Instructor: Date:

Learner: Regional Branch:

Program: Center:

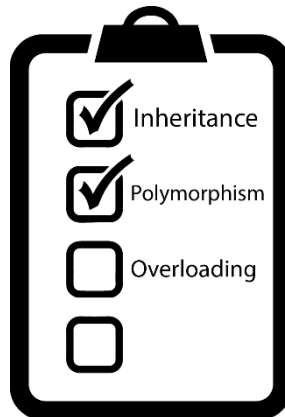
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

What features do we use for programming with OOP?



Learning Outcome:

Describe common activities at work to a using short sentences in familiar spoken and written situations.

Make the result report from the analysis of the information system, based on the customer requirements, following established rules and protocols.

Ask your partner:

- What programming features are you familiar with?
- When do you think software developers use inheritance, polymorphism, and overloading for?

Can Do Statements:

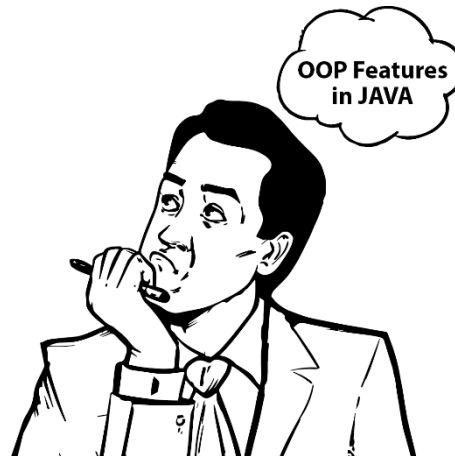
- I can talk about inheritance, polymorphism, and overloading.
- I can give specific information about inheritance, polymorphism, and overloading.
- I can recognize specific vocabulary related to inheritance, polymorphism, and overloading to use it in particular situations.



Let's Explore!

OOP Features* in JAVA.

*"Features" are qualities or characteristics.

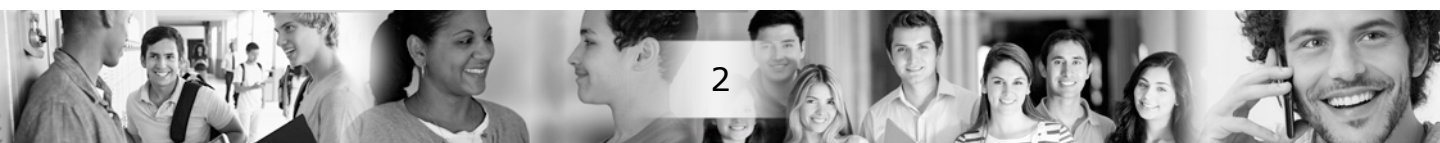


Instructions: Read the text below.

Manager: As software developers team leaders, you know that for OOP there are several important concepts. Today, I'm creating multiple inheritances; then, I'm going to call you to explain the next step so you can lead a session with your team. You don't have to declare the class, and you don't have to allow interfaces. You are only going to prepare them to use these concepts when they are designing their software in the production process.

You can choose when to do it. It's up to you. You need to remember we're going to focus on Java since it is an object-oriented language that provides the features of inheritance, polymorphism, and overloading. I know people doing this training don't have any knowledge of these features, so I'm going to explain these concepts. I'm going to begin with Inheritance. This is the mechanism by which an object adopts some or all properties of another object through hierarchical classification. With inheritance, many objects take more specialized variations from the main objects or classes. Mrs. Parker is working in a brochure about inheritance with key ideas this week, so it's going to be ready for your next team session.

Finally, we need to talk about polymorphism. This feature allows a defined variable with a type of data to take the values from secondary classes that inherit the main class. Therefore, one method needs to have multiple implementations, and every class has one or several methods. Now, if you talk about overloading, it means that two methods have the same name but take different input parameters; or it can have two outcomes, and it can calculate or provide more than one version depending on the context. Anyway, you don't need to worry about it because I'm knowledgeable about polymorphism. In fact, I'm writing a handout about it this afternoon.



**Let's Practice!****Talking About Inheritance and Polymorphism.**

Instructions: Use the words from the Word Bank to complete each sentence.

Word Bank

need to / have to / has to / have / needs to / need / mean to / mean

1. Inheritance doesn't _____ classify objects.
2. Polymorphism doesn't _____ to process data in a general way.
3. Overloading doesn't _____ that you need two different names for a method to take different input parameters.

**Let's Practice!****Vocabulary About Overloading, Polymorphism, and Inheritance.**

Instructions: Select the correct answer.

1. Thanks to the concept of classes, the programmer is going _____ new data types that need a particular definition.
 - a. to create
 - b. to transfer
 - c. to run
 - d. to use
2. This is something that is going to be done slowly in overloading with data.
 - a. to create
 - b. to process
 - c. to develop
 - d. to transfer
3. The Java Programming language is working in applications on _____.
 - a. networks
 - b. Internet
 - c. OPP languages
 - d. networks and Internet
4. Nowadays, it is being used to send a message from a program to the user.
 - a. OOP
 - b. polymorphism
 - c. network perimeter
 - d. dialog box
5. Programmers are reducing development time and are being more precise at coding due to this property of OPP.
 - a. object
 - b. inheritance
 - c. dialog box
 - d. tag
6. As we know, polymorphism is going to establish a defined variable with a type of data to take the values from _____ that inherit the main class.
 - a. main classes
 - b. all classes
 - c. secondary classes
 - d. no classes





Extension Activity

Talking About Polymorphism in OOP.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for more information about polymorphism in OOP.
- Write a short text describing how it works.
- Use the grammar and vocabulary studied in this lesson.
- Share your text with your classmates and instructor.

Self-Assessment

Yes

No

- I look for more information about polymorphism in OOP.
- I explain how it works through a brief composition.
- I use the grammar and vocabulary I learned in the lesson.
- I read my summary to my classmates and instructor.

☐☐☐☐☐☐☐☐

Target Words

- knowledgeable** (adjective): a person that knows a lot about something
e.g. Anyway, you don't need to worry about it because I'm knowledgeable about polymorphism.
- team session** (noun): meetings to train or lead a group about a specific topics
e.g. Mrs. Parker is working in a brochure about inheritance with key ideas this week, so it's going to be ready for your next team session.
- inheritance** (noun): characteristics that descend or come from something or someone
e.g. I'm going to begin with Inheritance. This is the mechanism by which an object adopts some or all properties of another object.
- several** (determiner): some; an amount that is not exact but is fewer than many
e.g. As software developers team leaders, you know that for OOP there are several important concepts.
- interface** (noun): a set of abstract methods and constants aimed to determine the functionality of a class
e.g. You don't have to allow interfaces.
- up to you** (idiom): to be someone else's responsibility to do something
e.g. You are only going to prepare them to use these concepts. You can choose when to do it. It's up to you.



Learning Points

Present Continuous and Be Going to in Affirmative.

Be going to and the **present continuous** are used to talk about future actions and events.

We use 'be **going to**' to express the future when we intend or decide to do something that we did not arrange before. It is just an intention. **Be going to** gives an extra emphasis on the idea of intention.

This is the structure for the affirmative form:

S + **be going to** + **main verb in base form** + complement

Examples:

*We **are going to develop** a new software the next month.*

*I **am going to fix** the network as soon as possible.*

On the other hand, we use the **present continuous** when we say what we planned and arranged to do at a specific time in the future. These are fixed plans with definite time and/or place.

Remember the structure for the affirmative form:

S + Auxiliary (**verb to be**) + main verb + **ing** + complement

Examples:

*We're **talking** about object oriented language the next class.*

*They **are designing** their software in the production process.*

Examples

- *I'm going to call you.*
- *You are only going to prepare them to use these concepts.*
- *Today, I'm creating multiple inheritances.*





Learning Points

Verbs with Infinitives in Negative.

An **infinitive** is a verb that is not conjugated, it means, it never goes with the suffixes s, es, or ing. Don't forget to use the word "to" in front of the verb, in this case, "to" is not a preposition but is a part of the verb.

Infinitives in Negative Form

When you want to use the negative form of the infinitive you can use the auxiliary verb of the grammar tense of your sentence (don't, doesn't, didn't), or you can add the word "**not to**" after the verb.

Examples:

- You can **choose to come** to the meeting in the morning or in the afternoon.
- You **don't have to decide** right now.
- He **doesn't need to come** to the office today.
- We **didn't want to redesign** the software.
- I prefer **not to talk** about the situation.

Examples

- You *don't have to declare the class.*
- You *don't have to allow interfaces*